



Univerza v Mariboru

Fakulteta za elektrotehniko,
računalništvo in informatiko

ORV – poročilo

Žiga Pešti, Anja Grudnik, Anže Žunec

Maribor, Junij 2026

Contents

Kateri model je bil uporabljen	4
Zakaj je bil izbran	4
Potek prepoznavе obraza	4
Kako je potekalo učenje	4
Kateri hiperparametri so bili optimizirani	5
Kako je potekala optimizacija	5
Katere metrike so bile uporabljene.....	5
Rezultati na testnih podatkih	5
Testiranje modela s testi	7
Test 1: Prepoznava pravilne osebe	7
Namen.....	7
Kako deluje.....	8
Rezultati.....	8
Zaključek.....	10
Test 2 : Zavrnitev napačnega uporabnika	10
Namen.....	10
Kako deluje.....	10
Rezultati.....	10
Zaključek.....	12
Test 3: Obnašanje modela brez obraza	12
Namen.....	12
Kako deluje.....	12
Rezultati.....	12
Zaključek.....	13
Test 4: Threshold verjetnosti	13
Namen.....	13
Kako deluje.....	13
Rezultati.....	13
Slike z nizko zaupnostjo:.....	13
Zaključek.....	14
Zaključek.....	14

Test 5: Hitrost modela	14
Namen	14
Kako deluje.....	14
Rezultati.....	14
Zaključek.....	15
Test 6: Več obrazov na sliki	15
Namen	15
Kako deluje.....	15
Rezultati.....	15
Zaključek	16
Threshold	17
Analiza različnih vrednosti thresholda	18
Analiza slik z nizko stopnjo zaupanja	19
Analiza vpliva osvetlitve na delovanje modela	20
Izboljšave modela na podlagi praktične uporabe	21
Zajem in organizacija podatkov.....	23
Predobdelava slik	23
Augmentacija podatkov	Error! Bookmark not defined.
Povezava z NPO mobilno aplikacijo.....	26
Končni rezultat	27
API, integracija sistema in priprava okolja za zagon	28
API endpointi	28
Validacija in obravnava napak	31
Docker in enostaven zagon	32
Testiranje API-ja	34

Kateri model je bil uporabljen

Za prepoznavo obrazov smo uporabili algoritem Support Vector Machine (SVM) z RBF jedrom. SVM je klasifikacijski algoritem strojnega učenja ki se nauči ločevati med različnimi razredi. V našem primeru med različnimi osebami.

Zakaj je bil izbran

SVM je bil izbran ker se dobro obnese na manjših naborih podatkov in ker skupaj s kosinusno podobnostjo tvori kombiniran pristop k prepoznavi. Medtem ko kosinusna podobnost primerja slike direktno, SVM pristopi k problemu kot klasifikacijska naloga.

Potek prepoznave obraza

Med prijavo sistem najprej zazna obraz na sliki. Nato se obraz obdela na enak način kot med učenjem modela in pošlje klasifikatorju SVM. Model vrne napoved uporabnika ter stopnjo zaupanja (verjetnost), na podlagi katere se odloči, ali je prepoznava dovolj zanesljiva. V aplikaciji je bil uporabljen prag zaupanja (threshold) 0,7. Če je bila napovedana verjetnost nižja od tega praga, je bila prijava zavrnjena.

Kako je potekalo učenje

Podatki so bili naloženi iz .npz datotek:

```
22  for npz_file in USERS_DIR.glob("*.npz"):
23      data = np.load(npz_file, allow_pickle=True)
24      samples = data["samples"]
25      username = str(data["username"])
```

Vsaka slika je bila pretvorjena v vektor in normalizirana z StandardScaler:

```
30  scaler = StandardScaler()
31  X_train = scaler.fit_transform(X_train)
```

Podatki so bili razdeljeni na učno (80%) in testno množico (20%).

Kateri hiperparametri so bili optimizirani

Optimizirani so bili trije hiperparametri:

- C , kaznovanje napak (0.1, 1, 10, 100)
- gamma , vpliv posameznega vzorca (scale, auto, 0.001, 0.01)
- kernel , tip jedra (rbf, linear)

Kako je potekala optimizacija

Za optimizacijo smo uporabili GridSearchCV ki preizkusi vse kombinacije parametrov s 3-kratno križno validacijo:

```
56 param_grid = {
57     'C': [0.1, 1, 10, 100],
58     'gamma': ['scale', 'auto', 0.001, 0.01],
59     'kernel': ['rbf', 'linear']
60 }
61 grid_search = GridSearchCV(SVC(probability=True), param_grid, cv=3, scoring='accuracy', verbose=1)
62 grid_search.fit(X_train, y_train)
```

Skupaj je bilo preizkušenih 32 kombinacij parametrov.

Katere metrike so bile uporabljene

- Accuracy , splošna točnost modela
- Confusion matrix , prikaz pravih in napačnih napovedi
- Precision, Recall, F1-score , podrobna evalvacija po vsaki osebi

Rezultati na testnih podatkih

Slika 1: Rezultati učenja SVM modela brez augmentacije. Model je bil naučen na 28 vzorcih (22 učnih, 6 testnih). Točnost na testnih podatkih je bila 66.67%. Opazimo da model Maje sploh ni prepoznal (precision 0.00, recall 0.00). Razlog je premajhno število učnih vzorcev in premajhna raznolikost slik. Model je Majo v vseh primerih napačno prepoznal kot Anjo, kar je razvidno iz confusion matrix. To je pokazalo potrebo po augmentaciji podatkov.

```

[STATUS] Skupaj uspešno naloženo: 28 vzorcev obrazov.
[INFO] Podatki razdeljeni: 22 za učenje, 6 za test.
[INFO] Normalizacija podatkov z StandardScaler...
[STATUS] Normalizacija uspešno izvedena!
[INFO] Optimizacija hiperparametrov z GridSearchCV...
Fitting 3 folds for each of 32 candidates, totalling 96 fits
[STATUS] Najboljši hiperparametri: {'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}
[STATUS] Najboljša cross-validation točnost: 67.86%
[EVALVACIJA] Točnost modela na testnih podatkih: 66.67%
[EVALVACIJA] Confusion matrix:
[[2 0 0]
 [2 0 0]
 [0 0 2]]
[EVALVACIJA] Classification report:
C:\Users\anjag\AppData\Local\Programs\Python\Python314\Lib\site-packages\sklearn\metrics\_classification.py:1833: UndefinedMetricWarning: Precision is ill-d
efined and being set to 0.0 in labels with no predicted samples. Use 'zero_division' parameter to control this behavior.
  _warn_prf(average, modifier, f'{metric.capitalize()} is', result.shape[0])
C:\Users\anjag\AppData\Local\Programs\Python\Python314\Lib\site-packages\sklearn\metrics\_classification.py:1833: UndefinedMetricWarning: Precision is ill-d
efined and being set to 0.0 in labels with no predicted samples. Use 'zero_division' parameter to control this behavior.
  _warn_prf(average, modifier, f'{metric.capitalize()} is', result.shape[0])
C:\Users\anjag\AppData\Local\Programs\Python\Python314\Lib\site-packages\sklearn\metrics\_classification.py:1833: UndefinedMetricWarning: Precision is ill-d
efined and being set to 0.0 in labels with no predicted samples. Use 'zero_division' parameter to control this behavior.
  _warn_prf(average, modifier, f'{metric.capitalize()} is', result.shape[0])
      precision    recall  f1-score   support

   anja         0.50      1.00      0.67         2
   maja         0.00      0.00      0.00         2
   ziga         1.00      1.00      1.00         2

 accuracy         0.67
 macro avg         0.50      0.67      0.56
weighted avg         0.50      0.67      0.56

[STATUS] Model, scaler in label_map uspešno shranjeni!

```

Slika 2: Rezultati učenja SVM modela z augmentacijo. Tukaja je prišlo do manjše neuspešnosti, saj augmentacija ni bila 100% uspešna, kar 38 od 292 slik je bilo preskočenih, ker face detection ni zaznal obraza. To se je zgodilo, ker transformacija preveč popači obraz, na primer pri preveč temnih slikah (`lowlight_inverse`) ali preveč zamegljenem obrazu. Te slike so bile samodejno izključene iz učne množice.

Rezultati učenja SVM modela z augmentiranimi podatki (254 vzorcev). Model je dosegel 92.16% točnost na testnih podatkih. Najboljši hiperparametri so bili $C=10$, $\gamma=\text{scale}$, $\text{kernel}=\text{rbf}$. Vse tri osebe so prepoznane z precision in recall nad 90%, confusion matrix pa pokaže le 3 napačne napovedi od 51.

```

s\ziga.npz (87 vzorcev)
[INFO] Preskocene slike:
- slika_714931d7_rot_p8.jpg: obraz ni zaznan
- Snapchat-1007503551_blur.jpg: obraz ni zaznan
- Snapchat-1177066886_lowlight_inverse.jpg: obraz ni zaznan
- Snapchat-1257223097_blur.jpg: obraz ni zaznan
- Snapchat-1257223097_bright_down.jpg: obraz ni zaznan
- Snapchat-1257223097_noise.jpg: obraz ni zaznan
- Snapchat-1257223097_rot_m8.jpg: obraz ni zaznan
- Snapchat-1257223097_rot_p8.jpg: obraz ni zaznan
- Snapchat-1257223097_zoom.jpg: obraz ni zaznan
- Snapchat-1762051350_noise.jpg: obraz ni zaznan
- Snapchat-2141482201_bright_up.jpg: obraz ni zaznan
- Snapchat-2141482201_flip.jpg: obraz ni zaznan
- Snapchat-326356315_zoom.jpg: obraz ni zaznan
- 10_lowlight_inverse.jpg: obraz ni zaznan
- 1_lowlight_inverse.jpg: obraz ni zaznan
- 2_lowlight_inverse.jpg: obraz ni zaznan
- 3_blur.jpg: obraz ni zaznan
- 3_bright_down.jpg: obraz ni zaznan
- 3_bright_up.jpg: obraz ni zaznan
- 3_flip.jpg: obraz ni zaznan
- 3_lowlight_gamma.jpg: obraz ni zaznan
- 3_lowlight_inverse.jpg: obraz ni zaznan
- 3_noise.jpg: obraz ni zaznan
- 3_rot_m8.jpg: obraz ni zaznan
- 3_zoom.jpg: obraz ni zaznan
- 4_lowlight_inverse.jpg: obraz ni zaznan
- 6_lowlight_inverse.jpg: obraz ni zaznan
- 6_rot_m8.jpg: obraz ni zaznan
- 8_lowlight_inverse.jpg: obraz ni zaznan
- 8_rot_p8.jpg: obraz ni zaznan
- 9_blur.jpg: obraz ni zaznan
- 9_bright_down.jpg: obraz ni zaznan
- 9_bright_up.jpg: obraz ni zaznan

```

```

- 9__bright_down.jpg: obraz ni zaznan
- 9__bright_up.jpg: obraz ni zaznan
- 9__flip.jpg: obraz ni zaznan
- 9__noise.jpg: obraz ni zaznan
- 9__rot_m8.jpg: obraz ni zaznan
- 9__rot_p8.jpg: obraz ni zaznan
- 9__zoom.jpg: obraz ni zaznan

```

```

ga-vida> python train_model.py
[STATUS] Skupaj uspešno naloženo: 254 vzorcev obrazov.
[INFO] Podatki razdeljeni: 203 za učenje, 51 za test.
[INFO] Normalizacija podatkov z StandardScaler...
[STATUS] Normalizacija uspešno izvedena!
[INFO] Optimizacija hiperparametrov z GridSearchCV...
Fitting 3 folds for each of 32 candidates, totalling 96 fits
[STATUS] Najboljši hiperparametri: {'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}
[STATUS] Najboljša cross-validation točnost: 91.15%
[EVALVACIJA] Točnost modela na testnih podatkih: 92.16%
[EVALVACIJA] Confusion matrix:
[[19  1  1]
 [ 1 12  0]
 [ 1  0 16]]
[EVALVACIJA] Classification report:

```

	precision	recall	f1-score	support
anja	0.90	0.90	0.90	21
maja	0.92	0.92	0.92	13
ziga	0.94	0.94	0.94	17
accuracy			0.92	51
macro avg	0.92	0.92	0.92	51
weighted avg	0.92	0.92	0.92	51

```

[STATUS] Model, scaler in label_map uspešno shranjeni!

```

Testiranje modela s testi

Verjetnost napovedi

Vsaka napoved modela je opremljena z verjetnostjo, ki pove kako prepričan je model v svojo odločitev. Vrednost 0.97 pomeni 97% prepričanost. Višja kot je verjetnost, zanesljivejša je napoved. V testih opazujemo tako točnost napovedi kot tudi verjetnosti, saj nizka verjetnost pri pravilni napovedi lahko kaže na problematične vzorce.

Test 1: Prepoznava pravične osebe

Namen

Preveriti ali model pravilno prepozna osebe, ki so registrirane v sistemu.

Kako deluje

Test naloži vse shranjene vzorce iz mape data/users/ in za vsakega pokliče predict_user(). Rezultat primerjamo z dejanskim uporabnikom in preverimo ali se ujema.

Rezultati

- Anja: 98.06% (101/103 vzorcev pravilno)
- Maja: 98.44% (63/64 vzorcev pravilno)
- Žiga: 98.85% (86/87 vzorcev pravilno)

Povprečna verjetnost je bila izračunana kot vsota vseh verjetnosti pravih napovedi deljena s številom vzorcev ($\text{avg} = \sum \text{verjetnosti} / n$). Pri Anji je povprečna verjetnost 0.97, najnižja pa 0.53, ena slika je bila prepoznana pravilno, ampak model ni bil prepričan v svojo odločitev. Pri Maji je povprečna verjetnost 0.97, najnižja 0.75, vse napovedi so bile nad 0.75, kar kaže na enakomerno prepričanost modela. Pri Žigi je povprečna verjetnost 0.98, najnižja 0.88, model je bil pri Žigi najbolj konsistenten od vseh treh.

Zaključek

Model je pravilno prepoznal registrirane uporabnike v 250 od 254 primerov. Točnost je pri vseh treh osebah nad 98%, kar potrjuje da model zanesljivo deluje na podatkih na katerih je bil naučen.

Test 2 : Zavrnitev napačnega uporabnika

Namen

Preveriti ali model zavrne dostop osebi, ki se poskuša prijaviti pod tujim imenom.

Kako deluje

Za vsakega uporabnika vzamemo slike preostalih dveh in preverimo ali jih model napačno prepozna kot prvega. Na primer: vzamemo Majine slike in preverimo ali model misli da je to Anja. Če odgovori ne, pomeni pravilna zavrnitev.

Rezultati

- Anja slike: preverja Maja: 12/12 ✓, povprečna verjetnost 0.85, najnižja 0.60
- Anja slike : preverja Žiga: 12/12 ✓, povprečna verjetnost 0.85, najnižja 0.60
- Maja slike : preverja Anja: 7/7 ✓, povprečna verjetnost 0.88, najnižja 0.55
- Maja slike : preverja Žiga: 7/7 ✓, povprečna verjetnost 0.88, najnižja 0.55
- Žiga slike : preverja Anja: 8/9 X, povprečna verjetnost 0.84, najnižja 0.57
- Žiga slike : preverja Maja: 9/9 ✓, povprečna verjetnost 0.85, najnižja 0.57

```
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.91)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.82)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.72)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.92)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.95)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.94)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.95)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.96)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.73)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.83)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.60)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.91)
[TEST] anja slike → preverja Ziga: 12/12 ✓
```

```
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.92)
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.96)
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.89)
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.94)
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.55)
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.94)
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.98)
[TEST] Maja slike → preverja anja: 7/7 ✓
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.92)
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.96)
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.89)
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.94)
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.55)
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.94)
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.98)
[TEST] Maja slike → preverja Ziga: 7/7 ✓
[PREDICT] Prepoznan uporabnik: ziga (verjetnost: 0.99)
[PREDICT] Prepoznan uporabnik: ziga (verjetnost: 0.98)
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.59)
[PREDICT] Prepoznan uporabnik: ziga (verjetnost: 0.74)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.37)
[PREDICT] Prepoznan uporabnik: ziga (verjetnost: 0.96)
[PREDICT] Prepoznan uporabnik: ziga (verjetnost: 0.91)
[PREDICT] Prepoznan uporabnik: ziga (verjetnost: 0.98)
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.57)
[TEST] Ziga slike → preverja anja: 8/9 X
```

```
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.91)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.82)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.72)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.92)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.95)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.94)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.95)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.97)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.73)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.83)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.60)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.91)
[TEST] anja slike → preverja Maja: 12/12 ✓
```

```
[PREDICT] Prepoznan uporabnik: ziga (verjetnost: 0.99)
[PREDICT] Prepoznan uporabnik: ziga (verjetnost: 0.98)
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.59)
[PREDICT] Prepoznan uporabnik: ziga (verjetnost: 0.74)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.37)
[PREDICT] Prepoznan uporabnik: ziga (verjetnost: 0.96)
[PREDICT] Prepoznan uporabnik: ziga (verjetnost: 0.99)
[PREDICT] Prepoznan uporabnik: ziga (verjetnost: 0.98)
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.57)
[TEST] Ziga slike → preverja Maja: 9/9 ✓
```

Pri Anjinih slikah je model prepoznal napačno osebo z povprečno verjetnostjo 0.85 , model je bil razmeroma prepričan da gre za napačno osebo, najnižja verjetnost je bila 0.60. Pri Majinih slikah je povprečna verjetnost 0.88, najnižja 0.55 , vse zavrnitve so bile pravilne. Pri Žigovih slikah je povprečna verjetnost 0.79, najnižja 0.37. Edina napaka se je zgodila pri sliki z verjetnostjo 0.37 , model ni bil prepričan, kar je privedlo do napačne klasifikacije.

Zaključek

Model je pravilno zavrnil napačnega uporabnika v 55 od 56 primerov. Edina napaka se je zgodila pri eni od Žigovih slik, kjer je model prepoznal obraz kot Majo z verjetnostjo 0.59, ki je najnižja zaupnost v celotnem testu. Skupna točnost zavrnitve je 98.21%.

Test 3: Obnašanje modela brez obraza

Namen

Preveriti kako se model obnaša ko dobi sliko brez obraza, črno sliko, belo sliko in naključen šum.

Kako deluje

Modelu podamo 3 slike ki ne vsebujejo obraza in preverimo ali model to zazna. Pričakujemo nizko verjetnost pri vseh treh.

Rezultati

- Črna slika: anja (verjetnost: 0.45)
- Bela slika: anja (verjetnost: 0.42)
- Naključen šum: anja (verjetnost: 0.44)

```
[TEST] Testiram obnašanje modela brez obraza...  
[PREDICT] Prepoznani uporabnik: anja (verjetnost: 0.45)  
[PREDICT] Prepoznani uporabnik: anja (verjetnost: 0.45)  
[PREDICT] Prepoznani uporabnik: anja (verjetnost: 0.42)  
[PREDICT] Prepoznani uporabnik: anja (verjetnost: 0.42)  
[PREDICT] Prepoznani uporabnik: anja (verjetnost: 0.44)  
[PREDICT] Prepoznani uporabnik: anja (verjetnost: 0.44)
```

```
[TEST] Nizka zaupnost ( $\{< 0.7\}$ ) pri: 3/3 primerih
```

```
[TEST OPOZORILO] Model ne zavrne slik brez obraza – priporočena je dodatna validacija!
```

Zaključek

Model slik brez obraza ne zavrne v vseh treh primerih poda napoved z verjetnostjo med 0.42 in 0.45. Zanimivo je da je model v vseh primerih prepoznal Anjo. To sicer ni naključje, saj ima največ učnih vzorcev (103) zato je model nanjo najbolj naučen in jo izbere kot privzeto napoved kadar ni prepričan. Verjetnosti so nizke kar kaže da model ni prepričan v svojo odločitev, ampak prijave ne zavrne sam od sebe. To se mi zdi da je ugotovitev ki kaže da bi bilo smiselno dodati threshold, torej mejo pod katero se prijava avtomatsko zavrne ne glede na napoved modela.

Test 4: Threshold verjetnosti

Namen

Preveriti koliko napovedi pade pod prag verjetnosti 0.7, ki smo ga določili kot minimalno mejo za zanesljivo prepoznavo.

Kako deluje

Test za vsako testno sliko pokliče `predict_user()` in preveri ali je verjetnost nad ali pod pragom 0.7. Slike z nizko verjetnostjo so izpisane posebej.

Rezultati

- Nad pragom (≥ 0.7): 23/28
- Pod pragom (< 0.7): 5/28

Slike z nizko zaupnostjo:

- Snapchat-628420678.jpg , anja (0.60)
- Snapchat-2141482201.jpg , maja (0.55)
- 2.JPG , maja (0.59)
- 4.JPG , anja (0.44)
- 9.jpg , maja (0.57)

Zaključek

5 od 28 slik pade pod prag 0.7. Te slike bi bile v aplikaciji zavrnjene. Večina je bila pravilno prepoznana ampak z nizko prepričanostjo, kar kaže da gre za slike slabše kakovosti ali neugodnega kota. Threshold 0.7 se izkaže kot smiselna meja.

```
[TEST] Testiram threshold verjetnosti (prag: 0.7)...  
[PREDICT] Prepoznani uporabnik: anja (verjetnost: 0.91)  
[PREDICT] Prepoznani uporabnik: anja (verjetnost: 0.82)  
[PREDICT] Prepoznani uporabnik: anja (verjetnost: 0.72)  
[PREDICT] Prepoznani uporabnik: anja (verjetnost: 0.92)  
[PREDICT] Prepoznani uporabnik: anja (verjetnost: 0.95)  
[PREDICT] Prepoznani uporabnik: anja (verjetnost: 0.94)  
[PREDICT] Prepoznani uporabnik: anja (verjetnost: 0.95)  
[PREDICT] Prepoznani uporabnik: anja (verjetnost: 0.97)  
[PREDICT] Prepoznani uporabnik: anja (verjetnost: 0.73)  
[PREDICT] Prepoznani uporabnik: anja (verjetnost: 0.83)  
[PREDICT] Prepoznani uporabnik: anja (verjetnost: 0.60)  
[NIZKA ZAUPNOST] Snapchat-628420678.jpg: anja (verjetnost: 0.60)  
[PREDICT] Prepoznani uporabnik: anja (verjetnost: 0.91)  
[PREDICT] Prepoznani uporabnik: maja (verjetnost: 0.92)  
[PREDICT] Prepoznani uporabnik: maja (verjetnost: 0.96)  
[PREDICT] Prepoznani uporabnik: maja (verjetnost: 0.89)  
[PREDICT] Prepoznani uporabnik: maja (verjetnost: 0.94)  
[PREDICT] Prepoznani uporabnik: maja (verjetnost: 0.55)  
[NIZKA ZAUPNOST] Snapchat-2141482201.jpg: maja (verjetnost: 0.55)  
[PREDICT] Prepoznani uporabnik: maja (verjetnost: 0.94)  
[PREDICT] Prepoznani uporabnik: maja (verjetnost: 0.98)  
[PREDICT] Prepoznani uporabnik: ziga (verjetnost: 0.99)  
[PREDICT] Prepoznani uporabnik: ziga (verjetnost: 0.98)  
[PREDICT] Prepoznani uporabnik: maja (verjetnost: 0.59)  
[NIZKA ZAUPNOST] 2.JPG: maja (verjetnost: 0.59)  
[PREDICT] Prepoznani uporabnik: ziga (verjetnost: 0.74)  
[PREDICT] Prepoznani uporabnik: anja (verjetnost: 0.44)  
[NIZKA ZAUPNOST] 4.JPG: anja (verjetnost: 0.44)  
[PREDICT] Prepoznani uporabnik: ziga (verjetnost: 0.96)  
[PREDICT] Prepoznani uporabnik: ziga (verjetnost: 0.99)  
[PREDICT] Prepoznani uporabnik: ziga (verjetnost: 0.98)  
[PREDICT] Prepoznani uporabnik: maja (verjetnost: 0.57)  
[NIZKA ZAUPNOST] 9.jpg: maja (verjetnost: 0.57)
```

```
[TEST] Nad pragom: 23/28  
[TEST] Pod pragom: 5/28
```

Zaključek

5 od 28 slik pade pod prag 0.7. Te slike bi bile v aplikaciji zavrnjene. Večina je bila pravilno prepoznana ampak z nizko prepričanostjo, kar kaže da gre za slike slabše kakovosti ali neugodnega kota. Threshold 0.7 se izkaže kot smiselna meja.

Test 5: Hitrost modela

Namen

Preveriti ali je model dovolj hiter za realnočasovno uporabo v aplikaciji.

Kako deluje

Modelu podam 100 naključnih slik in za vsako posebej izmerimo čas napovedi. Na koncu izpišem najhitrejšo, najpočasnejšo in povprečno napoved.

Rezultati

- Najhitrejša napoved: 40.80ms
- Najpočasnejša napoved: 1013.76ms
- Povprečni čas na napoved: 55.50ms

```
[TEST] Najhitrejša napoved: 41.12ms  
[TEST] Najpočasnejša napoved: 1329.37ms  
[TEST] Povprečni čas na napoved: 114.72ms  
[TEST OPOZORILO] Model je počasen za realnočasovno uporabo.
```

Zaključek

Povprečni čas napovedi je 55.50ms kar je dovolj hitro za realnočasovno uporabo. Najpočasnejša napoved je bila 1013.76ms ta skok je verjetno posledica prvega klica ko model nalaga iz diska, vse nadaljnje napovedi so bile bistveno hitrejše.

V aplikaciji se prepoznavanje obrazov izvaja v realnem času med prijavo, torej mora model podati napoved dovolj hitro da uporabnik ne čaka. Če bi napoved trajala npr. 2-3 sekunde, bi bila uporabniška izkušnja slaba.

Test 6: Več obrazov na sliki

Namen

Preveriti kako se model obnaša ko slika vsebuje več obrazov hkrati, torej kateri obraz bo model uporabil za napoved.

Kako deluje

Test preveri vse testne slike in poišče tiste kjer face detection zazna več kot en obraz. Za vsak zaznani obraz pokliče `predict_user()` in izpiše napoved.

Rezultati

- Slike z več obrazi: 14/31
- Največje število zaznanih obrazov: 8 (Snapchat-1366706072.jpg)
- Pri slikah z več obrazi model napove za vsakega posebej


```

[VEČ OBRAZOV] 20250701_114723.jpg: 5 obrazi zaznani
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.91)
  → Obraz 1: anja (verjetnost: 0.91)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.55)
  → Obraz 2: anja (verjetnost: 0.55)
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.55)
  → Obraz 3: maja (verjetnost: 0.55)
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.52)
  → Obraz 4: maja (verjetnost: 0.52)
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.69)
  → Obraz 5: maja (verjetnost: 0.69)
[VEČ OBRAZOV] Screenshot_20260530_071542_Gallery.jpg: 5 obrazi zaznani
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.72)
  → Obraz 1: anja (verjetnost: 0.72)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.29)
  → Obraz 2: anja (verjetnost: 0.29)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.31)
  → Obraz 3: anja (verjetnost: 0.31)
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.59)
  → Obraz 4: maja (verjetnost: 0.59)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.49)
  → Obraz 5: anja (verjetnost: 0.49)
[VEČ OBRAZOV] Snapchat-1249357458.jpg: 2 obrazi zaznani
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.97)
  → Obraz 1: anja (verjetnost: 0.97)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.96)
  → Obraz 2: anja (verjetnost: 0.96)
[VEČ OBRAZOV] Snapchat-1366706072.jpg: 8 obrazi zaznani
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.63)
  → Obraz 1: anja (verjetnost: 0.63)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.73)
  → Obraz 2: anja (verjetnost: 0.73)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.56)
  → Obraz 3: anja (verjetnost: 0.56)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.55)
  → Obraz 4: anja (verjetnost: 0.55)
[PREDICT] Prepoznan uporabnik: ziga (verjetnost: 0.99)
  → Obraz 5: ziga (verjetnost: 0.99)
[PREDICT] Prepoznan uporabnik: ziga (verjetnost: 0.99)
  → Obraz 6: ziga (verjetnost: 0.99)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.46)
  → Obraz 7: anja (verjetnost: 0.46)
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.80)
  → Obraz 8: maja (verjetnost: 0.80)
[VEČ OBRAZOV] 4.JPG: 2 obrazi zaznani
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.44)
  → Obraz 1: anja (verjetnost: 0.44)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.37)
  → Obraz 2: anja (verjetnost: 0.37)
[VEČ OBRAZOV] 7.jpg: 2 obrazi zaznani
[PREDICT] Prepoznan uporabnik: ziga (verjetnost: 0.99)
  → Obraz 1: ziga (verjetnost: 0.99)
[PREDICT] Prepoznan uporabnik: ziga (verjetnost: 0.91)
  → Obraz 2: ziga (verjetnost: 0.91)

[TEST] Slike z več obrazi: 14/31
[TEST OPOZORILO] Aplikacija mora določiti kateri obraz uporabiti pri prijavi!

```

Zaključek

14 od 31 testnih slik vsebuje več zaznanih obrazov. Model obdela vsakega posebej ampak ne ve kateri je pravilen to je potencialna težava pri prijavi. Aplikacija bi morala vzeti samo prvi oz. največji obraz na sliki, kar je obraz ki je najbližje kameri. Izboljšave modela na podlagi testov

Threshold

```
[TEST] Testiram threshold verjetnosti (prag: 0.7)...
```

Verjetnost	Uporabnik
0.91	anja
0.82	anja
0.72	anja
0.92	anja
0.95	anja
0.94	anja
0.95	anja
0.97	anja
0.73	anja
0.83	anja
0.60	anja

[NIZKA ZAUPNOST] Snapchat-628420678.jpg: anja (verjetnost: 0.60)

Verjetnost	Uporabnik
0.91	anja
0.92	maja
0.96	maja
0.89	maja
0.94	maja
0.55	maja

[NIZKA ZAUPNOST] Snapchat-2141482201.jpg: maja (verjetnost: 0.55)

Verjetnost	Uporabnik
0.94	maja
0.98	maja
0.99	ziga
0.98	ziga
0.59	maja
0.74	ziga
0.44	anja
0.44	anja
0.96	ziga
0.99	ziga
0.98	ziga
0.57	maja
0.57	maja

```
[PREDICT] Prepoznan uporabnik: ziga (verjetnost: 0.99)
→ Obraz 3: ziga (verjetnost: 0.99)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.46)
→ Obraz 4: anja (verjetnost: 0.46)
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.80)
→ Obraz 5: maja (verjetnost: 0.80)
[VEČ OBRAZOV] 4.JPG: 2 obrazi zaznani
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.44)
→ Obraz 1: anja (verjetnost: 0.44)
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.37)
→ Obraz 2: anja (verjetnost: 0.37)
[VEČ OBRAZOV] 7.jpg: 2 obrazi zaznani
[PREDICT] Prepoznan uporabnik: ziga (verjetnost: 0.99)
→ Obraz 1: ziga (verjetnost: 0.99)
[PREDICT] Prepoznan uporabnik: ziga (verjetnost: 0.91)
→ Obraz 2: ziga (verjetnost: 0.91)

[TEST] Slike z več obrazi: 14/31
[TEST OPOZORILO] Aplikacija mora določiti kateri obraz uporabiti pri prijavi!
```

Zgornja slika prikazuje podatke prejšnjega testa pred izboljšavo tresholda. In tukaj se opazi problem, da kar 5 slik, ki ima nizko zaupnost, jih model ni zavrnil ampak le izpisuje opozorilo. Model jih sicer prepozna ampak ni mogel zanesljivo določiti kateri od treh uporabnikov je.

```
[PREDICT] Prepoznan uporabnik: ziga (verjetnost: 0.98)
[INFO] Model naložen iz predpomnilnika.
[PREDICT] Zavrtnjena napoved – prenizka zaupnost: 0.57
[NIZKA ZAUPNOST] 9.jpg: None (verjetnost: 0.57)

[TEST] Nad pragom: 23/28
[TEST] Pod pragom: 5/28

[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.92)
[INFO] Model naložen iz predpomnilnika.
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.95)
[INFO] Model naložen iz predpomnilnika.
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.94)
[INFO] Model naložen iz predpomnilnika.
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.95)
[INFO] Model naložen iz predpomnilnika.
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.97)
[INFO] Model naložen iz predpomnilnika.
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.73)
[INFO] Model naložen iz predpomnilnika.
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.83)
[INFO] Model naložen iz predpomnilnika.
[PREDICT] Zavrtnjena napoved – prenizka zaupnost: 0.60
[NIZKA ZAUPNOST] Snapchat-628420678.jpg: None (verjetnost: 0.60)
[INFO] Model naložen iz predpomnilnika.
[PREDICT] Prepoznan uporabnik: anja (verjetnost: 0.91)
[INFO] Model naložen iz predpomnilnika.
[PREDICT] Prepoznan uporabnik: maja (verjetnost: 0.92)
[INFO] Model naložen iz predpomnilnika.
```

Po dodanem thresholdu se rezultati testa niso bistveno spremenili, saj še vedno je 5 slik pod pragom 0.7. Razlika je v tem kaj model naredi s temi slikami. Prej je model vrnil uporabniško ime tudi pri nizki zaupnosti, torej bi po tem takem bila prijava uspešna kljub

temu da model ni bil prepričan. Zdaj pa model vrne None kar pomeni da se prijava zavrne. To je razvidno iz outputa kjer piše "Zavrnjena napoved - prenizka zaupnost" in None namesto uporabniškega imena.

Poleg tega je vidno da cache deluje. Prvi klic izpiše "Model naložen iz diska", vsi nadaljnji pa "Model naložen iz predpomnilnika", kar pomeni da se model ne nalaga vsakič znova.

Analiza različnih vrednosti thresholda

V prejšnji izboljšavi sem uvedla threshold 0,7, saj sem med testiranjem opazila, da model pri nekaterih slikah sicer pravilno prepozna uporabnika, vendar je stopnja zaupanja zelo nizka. V takšnih primerih napoved ni dovolj zanesljiva za uporabo v prijavnem sistemu, zato sem želela preveriti, kako izbira thresholda vpliva na delovanje modela.

Namen testa je bil ugotoviti, koliko prijav bi bilo sprejetih oziroma zavrnjenih pri različnih vrednostih thresholda ter katera vrednost predstavlja najboljše ravnovesje med varnostjo in uporabnostjo sistema.

Preizkusila sem vrednosti 0,5, 0,6, 0,7, 0,8 in 0,9. Za vsako vrednost sem preverila, koliko slik je bilo uspešno sprejetih in koliko jih je sistem zavrnil zaradi prenizke stopnje zaupanja.

===== ANALIZA THRESHOLDA =====		
Threshold 0.5	Sprejete: 26/28	Zavrnjene: 2/28
Threshold 0.6	Sprejete: 22/28	Zavrnjene: 6/28
Threshold 0.7	Sprejete: 21/28	Zavrnjene: 7/28
Threshold 0.8	Sprejete: 19/28	Zavrnjene: 9/28
Threshold 0.9	Sprejete: 16/28	Zavrnjene: 12/28

Pri thresholdu 0,5 je sistem sprejel 26 od 28 slik in zavrnil le 2. Takšna nastavev omogoča več uspešnih prijav, vendar hkrati poveča možnost, da sistem sprejme tudi manj zanesljive napovedi.

Z višanjem thresholda se število zavrnjenih prijav povečuje. Pri vrednosti 0,8 je bilo zavrnjenih že 9 od 28 slik, pri vrednosti 0,9 pa kar 12 od 28 slik. Sistem postane tako bistveno strožji, saj za uspešno prijavo zahteva zelo visoko stopnjo zaupanja.

Rezultati kažejo, da izbira thresholda pomembno vpliva na delovanje sistema. Nižji threshold omogoča več uspešnih prijav, višji threshold pa poveča varnost, saj zavrne več napovedi z nižjo stopnjo zaupanja.

Na podlagi rezultatov sem ocenila, da threshold 0,7 predstavlja najboljše ravnovesje med varnostjo in uporabnostjo. Pri tej vrednosti je bilo sprejetih 21 od 28 slik, hkrati pa

so bile zavrnjene napovedi z najnižjo stopnjo zaupanja. Zato sem threshold 0,7 obdržala tudi v končni različici aplikacije.

Analiza slik z nizko stopnjo zaupanja

Po uvedbi thresholda 0,7 sem želela podrobneje analizirati slike, pri katerih je model dosegel najnižjo stopnjo zaupanja. Namen analize je bil ugotoviti, katere značilnosti slik vplivajo na zmanjšanje zanesljivosti napovedi in povzročajo zavrnitev prijave.

Na spodnji sliki so prikazani primeri slik, pri katerih je bila izračunana verjetnost nižja od izbranega thresholda 0,7. V takšnih primerih sistem napoved samodejno zavrne in uporabniku ne dovoli prijave.

```
[PREDICT] Zavrnjena napoved – prenizka zaupnost: 0.60  
Datoteka: Snapchat-628420678.jpg  
Pravi uporabnik: anja  
Napoved modela: None  
Verjetnost: 0.60
```

```
[PREDICT] Zavrnjena napoved – prenizka zaupnost: 0.55  
Datoteka: Snapchat-2141482201.jpg  
Pravi uporabnik: Maja  
Napoved modela: None  
Verjetnost: 0.55
```

```
[PREDICT] Zavrnjena napoved – prenizka zaupnost: 0.59  
Datoteka: 2.JPG  
Pravi uporabnik: Ziga  
Napoved modela: None  
Verjetnost: 0.59
```

```
[PREDICT] Zavrnjena napoved – prenizka zaupnost: 0.44  
Datoteka: 4.JPG  
Pravi uporabnik: Ziga  
Napoved modela: None  
Verjetnost: 0.44
```

```
[PREDICT] Zavrnjena napoved – prenizka zaupnost: 0.57  
Datoteka: 9.jpg  
Pravi uporabnik: Ziga  
Napoved modela: None  
Verjetnost: 0.57
```

Rezultati so pokazali, da je bilo pod thresholdom skupno 5 slik. Pri uporabnici Anji je bila zaznana ena slika z verjetnostjo 0.60, pri uporabnici Maji ena slika z verjetnostjo 0.55, pri uporabniku Žigi pa tri slike z verjetnostmi 0.59, 0.57 in 0.44.

Po pregledu zavrnjenih slik sem ugotovila, da imajo te slike določene značilnosti, ki vplivajo na delovanje modela. Med njimi so različni koti obraza, drugačni izrazi obraza, delna zakritost obraza, večja oddaljenost od kamere in manj ugodni svetlobni pogoji. Najnižjo izračunano verjetnost je dosegla slika »4.JPG« uporabnika Žige, pri kateri je bila vrednost le 0,44.

Analiza je pokazala, da model pri določenih slikah doseže nižjo stopnjo zaupanja tudi takrat, ko gre za uporabnike, ki so vključeni v učno množico. To pomeni, da na zanesljivost napovedi poleg identitete uporabnika pomembno vplivajo tudi pogoji zajema slike ter kakovost zaznanega obraza.

Na podlagi rezultatov lahko zaključim, da model v večini primerov uspešno prepozna uporabnike, vendar se njegova stopnja zaupanja zmanjša pri slikah z večjimi odstopanji v osvetlitvi, položaju obraza ali zakritosti obraznih značilnosti.

Analiza vpliva osvetlitve na delovanje modela

Po analizi slik z nizko stopnjo zaupanja sem opazila, da je bilo več zavrnjenih slik posnetih v manj ugodnih svetlobnih pogojih. Zato sem želela preveriti, kako svetlobni pogoji vplivajo na zanesljivost delovanja modela.

Za izvedbo testa sem uporabila enake testne slike kot pri prejšnjih analizah. Iz vsake slike sem ustvarila dodatno različico z zmanjšano svetlostjo in nato primerjala povprečno verjetnost napovedi pri normalnih ter umetno potemnjenih slikah.

```
Povprečna verjetnost (normalne slike): 0.84
Povprečna verjetnost (temne slike): 0.52
Razlika: 0.32
```

Rezultati so pokazali, da je model pri normalnih slikah dosegel povprečno verjetnost 0,84, pri umetno potemnjenih slikah pa le 0,52. Povprečna stopnja zaupanja se je tako zmanjšala za 0,32.

Iz rezultatov je razvidno, da imajo svetlobni pogoji pomemben vpliv na delovanje modela. Pri temnejših slikah se stopnja zaupanja občutno zmanjša, zaradi česar sistem pogosteje zavrne prijavo. To potrjuje tudi ugotovitve iz prejšnjih analiz, saj so bile med problematičnimi slikami pogosto prisotne sence, slabša osvetlitev ali neenakomerno osvetljen obraz.

Na podlagi izvedenega testa lahko zaključim, da je osvetlitev eden izmed najpomembnejših dejavnikov, ki vplivajo na uspešnost prepoznav obrazov v razvitem

sistemu. Pri praktični uporabi je zato pomembno zagotoviti čim bolj enakomerne svetlobne pogoje, saj se v nasprotnem primeru zanesljivost napovedi občutno zmanjša.

Izboljšave modela na podlagi praktične uporabe

Po izvedenih testih sem sistem preizkusila tudi v realnem času z uporabo spletne kamere. Namen tega preizkusa je bil preveriti, ali model deluje enako dobro tudi pri dejanski uporabi in ne samo na pripravljenih testnih podatkih.

Med testiranjem sem opazila nepričakovano težavo. Kljub temu da je model na testnih podatkih dosegel 97,83 % točnost, je uporabnico Anjo pri prijavi pogosto prepoznal kot Majo ali Žigata. Pri tem je model dosegal zelo visoke vrednosti zaupanja (med 0,90 in 0,99), kar pomeni, da ni šlo zgolj za negotovost modela, ampak za sistematično napačno klasifikacijo.

[PREDICT] Ziga (0.57)	[PREDICT] anja (0.61)
[PREDICT] Ziga (0.55)	[PREDICT] anja (0.51)
[PREDICT] Ziga (0.58)	[PREDICT] anja (0.61)
[PREDICT] Ziga (0.59)	[PREDICT] anja (0.50)
[PREDICT] Ziga (0.53)	[PREDICT] maja (0.97)
[PREDICT] Ziga (0.54)	[PREDICT] maja (0.98)
[PREDICT] Ziga (0.55)	[PREDICT] maja (0.97)
[PREDICT] Ziga (0.52)	[PREDICT] maja (0.98)
[PREDICT] Ziga (0.57)	[PREDICT] maja (0.97)
[PREDICT] Ziga (0.59)	[PREDICT] maja (0.98)
[PREDICT] Ziga (0.58)	[PREDICT] maja (0.99)
[PREDICT] Ziga (0.58)	[PREDICT] maja (0.98)

Na zgornji sliki je prikazan primer izpisa iz terminala, kjer model več zaporednih slik uporabnice Anje prepozna kot Majo ali pa kot Žigata.

Za odpravo težave sem analizirala, na kakšnih slikah je bil model učen.

Opazila sem, da sem zaradi želje po prepoznavi tudi s sončnimi očali v učno množico vključila preveliko število slik s sončnimi očali glede na število preostalih fotografij. Zaradi tega se je model preveč prilagodil tem značilnostim in je pri nekaterih slikah slabše prepoznal moj obraz.

Za izboljšanje rezultatov sem v učno množico dodala približno 20 novih originalnih fotografij brez očal, posnetih v različnih pogojih. S tem sem povečala raznolikost

podatkov in izboljšala ravnovesje med posameznimi tipi slik. Iz naslednjih rezultatov je razvidno, da se je delovanje modela bistveno izboljšalo.

```
[PREDICT] anja (0.76)
[PREDICT] anja (0.77)
[PREDICT] anja (0.76)
[PREDICT] anja (0.78)
[PREDICT] anja (0.74)
[PREDICT] anja (0.75)
[PREDICT] anja (0.77)
[PREDICT] anja (0.72)
[PREDICT] anja (0.78)
[PREDICT] anja (0.82)
```

Za preverjanje, ali je dodajanje novih fotografij dejansko izboljšalo delovanje sistema, sem podatke ponovno izvozila, pripravila novo učno množico in model ponovno naučila. Nato sem primerjala rezultate nove različice z rezultati prejšnjega modela.

Po izvozu podatkov sem pridobila 18 veljavnih vzorcev za uporabnico Anjo, 72 vzorcev za uporabnico Majo, 94 vzorcev za uporabnika Žigo in 5 vzorcev za uporabnika Anžeta.

Model sem nato ponovno naučila na novi učni množici. Skupaj je bilo uporabljenih 189 vzorcev obrazov. Nova različica modela je dosegla 92,11 % točnost na testnih podatkih.

Čeprav je bila izmerjena točnost nekoliko nižja od prejšnje različice modela (97,83 %), je bilo delovanje pri dejanski uporabi stabilnejše. Na podlagi tega sem ugotovila, da večja količina podatkov sama po sebi ne zagotavlja boljšega modela. Pomembnejša sta raznolikost in kakovost originalnih podatkov.

Po nadaljnjem testiranju sistema v realnem času sem razširila učno množico z več originalnimi fotografijami uporabnice Anje, posnetimi v različnih pogojih (različne osvetlitve, zorni koti, očala in izrazi obraza). Nato sem model ponovno naučila in primerjala rezultate s prejšnjo različico.

Po dodatnem zbiranju podatkov je bilo za učenje uporabljenih 1384 vzorcev obrazov. Novi model je dosegel 97,83 % točnost na testni množici. Najboljši hiperparametri so bili $C = 10$, $\text{gamma} = \text{scale}$ in $\text{kernel} = \text{rbf}$. Cross-validation točnost je znašala 97,47 %.

Rezultati potrjujejo, da je povečanje števila in raznolikosti vzorcev pomembno izboljšalo sposobnost modela za prepoznavo obrazov. Posebej se je izboljšala prepoznavna uporabnice Anje, saj je model začel pravilno prepoznavati tudi slike z očali, različnimi izrazi obraza in različnimi svetlobnimi pogoji.

Zajem in organizacija podatkov

Pripravil sem začetni zajem slike za 2FA in testiranje kamere v datoteki main.py. Dodal sem postopek za zaznavo in izrez obraza iz vhodne slike, najprej prek skripte detect-face.py, kasneje pa prek funkcij v face_name_preview.py in api_server.py. Uredil sem strukturo podatkov v mapah data/test_images/lmeOsebe, kjer so slike razdeljene po uporabnikih. V repozitoriju so med drugim mape Ziga, Anze, Maja in anja. Pri pripravi podatkov sem uporabljal manifest predobdelave, ki beleži izvor slike, izhodno datoteko, okvir obraza, velikost slike in izvedene korake predobdelave.

```
osnove-racunalniskiga-vida > main.py > ...
1  import cv2
2  import os
3  import uuid
4
5  def capture_images(save_path="data/raw/"):
6      cap = cv2.VideoCapture(0)
7      print("Pritisni SPACE za zajem slike, ali 'q' za izhod.")
8
9      while True:
10         ret, frame = cap.read()
11         if not ret:
12             break
13
14         cv2.imshow("Zajem za 2FA Hribovc", frame)
15         key = cv2.waitKey(1) & 0xFF
16
17         if key == ord(' '):
18             img_name = f"{save_path}/slika_{uuid.uuid4().hex[:8]}.jpg"
19             cv2.imwrite(img_name, frame)
20             print(f"Shranjeno: {img_name}")
21         elif key == ord('q'):
22             break
23
24     cap.release()
25     cv2.destroyAllWindows()
26
27 if __name__ == "__main__":
28     capture_images()
29
```

Predobdelava slik

Augment_images.py je namenjen pripravi in razširitvi podatkovnega nabora, medtem ko se zaznava obraza izvaja v face_name_preview.py oziroma prek ORV API-ja v api_server.py.

Glavni koraki so:

- pretvorba slike v sivinski barvni prostor, ker model dela z obraznimi intenzitetami in ne potrebuje RGB informacije;
- zaznava največjega obraza z OpenCV Haar Cascade detektorjem;
- izrez relevantnega dela slike, torej obraza, namesto obdelave celotnega ozadja;

- sprememba velikosti obraza na standardizirano dimenzijo 128 x 128 pik;
- izravnava histograma za bolj primerljive kontraste med slikami;
- normalizacija vrednosti pik na interval od 0 do 1;
- zameglitev ozadja, ki ni obraz, pri predogledu in prijavi, da je poudarek na relevantnem delu slike.

```
def is_low_light(image, threshold):
    stats = brightness_stats(image)
    return stats["mean"] < threshold or stats["p10"] < threshold * 0.45, stats

def gamma_lighten(image, gamma=0.55):
    table = np.array([
        ((i / 255.0) ** gamma) * 255
        for i in range(256)
    ]).astype("uint8")
    return cv2.LUT(image, table)

def clahe_lab(image):
    lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)
    l_channel, a_channel, b_channel = cv2.split(lab)
    clahe = cv2.createCLAHE(cliplimit=2.0, tileGridSize=(8, 8))
    enhanced_l = clahe.apply(l_channel)
    enhanced_lab = cv2.merge((enhanced_l, a_channel, b_channel))
    return cv2.cvtColor(enhanced_lab, cv2.COLOR_LAB2BGR)

def inverse_contrast(image):
    inverted = cv2.bitwise_not(image)
    return cv2.addWeighted(image, 0.35, inverted, 0.65, 0)
```

Predogled obraza

Datoteka `face_name_preview.py` predstavlja osrednji del mojega ORV modula za zaznavo in prepoznavo obraza. V njej sem združil pripravo obraza, učenje oziroma uporabo modela, predogled kamere in logiko za prijavo uporabnika z obrazno prepoznavo.

Skripta najprej iz slike ali video toka kamere pripravi obraz v standardizirani obliki. Vhodna slika se pretvori v sivinski barvni prostor, nato se z OpenCV Haar Cascade detektorjem poišče obraz. Iz slike se izreže največji zaznani obraz, ta se spremeni na enotno velikost, izvede se izravnava histograma, vrednosti pik pa se normalizirajo.

V datoteko sem dodal tudi izboljšave za zahtevnejše pogoje, na primer slabšo osvetlitev. Za temne slike se uporabljajo dodatni postopki, kot sta gamma popravek in CLAHE, kar izboljša kontrast obraza. Dodan je bil tudi night mode, ki omogoča boljšo zaznavo obraza pri slabši svetlobi.

face_name_preview.py omogoča tudi učenje modela iz slik, ki so organizirane po uporabnikih v mapah data/test_images/lmeOsebe. Iz pripravljenih obrazov se ustvarijo značilke, model pa se nato shrani za kasnejšo uporabo pri prepoznavi. Pri prijavi skripta iz kamere sproti zazna obraz, ga pripravi na enak način kot učne slike in preveri, ali se ujema s pričakovanim uporabnikom.

Pomemben del skripte je tudi aplikacijska logika za 2FA prijavo. Pri prijavi z obrazom skripta ne vrne samo slike ali zaznave, ampak strukturiran rezultat, ki pove, ali je bila prijava uspešna, kateri uporabnik je bil prepoznan in s kakšno zanesljivostjo. Ta rezultat lahko backend uporabi za odločitev, ali se uporabniku dovoli nadaljevanje prijave v spletno aplikacijo.

V skripti sem dodal tudi več praktičnih izboljšav za demonstracijo in uporabo, prikaz imena zaznanega uporabnika ob obrazu, preverjanje praga zaupanja, avtomatsko potrjevanje uspešne zaznave, preverjanje odprtih oči ter zameglitev ozadja, ki ni obraz. S tem je rešitev bolj uporabna, pregledna in primerna za prikaz končnega delovanja sistema.

```
def draw_label(frame, box, name, confidence, threshold):
    x, y, w, h = box
    accepted = confidence >= threshold
    color = (0, 180, 0) if accepted else (0, 165, 255)
    display_name = name if accepted else f"{name}?"
    label = f"{display_name} podobnost: {confidence:.2f}"

    cv2.rectangle(frame, (x, y), (x + w, y + h), color, 2)

    label_y = max(28, y - 10)
    cv2.putText(
        frame,
        label,
        (x, label_y),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.7,
        color,
        2,
    )
```

```
def cosine_similarity(a, b):
    denominator = np.linalg.norm(a) * np.linalg.norm(b)
    if denominator == 0:
        return 0.0

    return float(np.dot(a, b) / denominator)

def predict_nearest(face, recognizer):
    face_flat = face.flatten()
    samples = recognizer["samples"]
    labels = recognizer["labels"]
    label_map = recognizer["label_map"]

    scores = [cosine_similarity(face_flat, sample) for sample in samples]
    best_index = int(np.argmax(scores))
    best_label = int(labels[best_index])

    return label_name(label_map, best_label), float(scores[best_index])
```

Povezava z NPO mobilno aplikacijo

Mobilna aplikacija je bila uporabljena kot dodaten vhodni kanal za ORV 2FA, kjer telefon zajame sliko obraza in jo pošlje nazaj v sistem. Ta del povezuje računalniški vid, MQTT komunikacijo, backend in uporabniški vmesnik.

- V spletni prijavi sem dodal izbiro med PC kamero in kamero telefona.
- Backend ustvari ORV 2FA challenge z identifikatorjem, pričakovanim uporabnikom, pragom ujemanja in časom poteka.
- Zahteva se prek MQTT objavi na topic hribovc/orv-2fa/request, na katerega se naroči mobilna aplikacija.
- Mobilna aplikacija z expo-camera prikaže ORV 2FA modal, zahteva dovoljenje za kamero, zajema sliko oziroma preview tok in pošlje base64 sliko na backend.
- Backend prejete slike pošlje naprej v ORV API, spremlja statusse pending, approved, rejected in expired ter spletni prijavi omogoča polling statusa.
- Kasnejši popravki so uredili dovoljenja kamere, stream namesto ene same slike, izbiro uporabnika in bolj stabilen telefonski login.

Končni rezultat

Končni rezultat mojega dela je ORV modul, ki iz slik pripravi standardizirane obraze, razširi podatkovni nabor z augmentacijo, nauči model za prepoznavo uporabnika in rezultat izpostavi prek API-ja. Rešitev ni ostala samo laboratorijski primer, ampak je integrirana v aplikacijski prijavni tok kot 2FA preverjanje. Z NPO mobilno aplikacijo je sistem dobil še možnost, da se obraz potrdi s telefonsko kamero, kar projekt poveže v celoto: spletna aplikacija, backend, MQTT komunikacija, mobilna aplikacija in računalniški vid.

API, integracija sistema in priprava okolja za zagon

Za povezavo modela za prepoznavo obrazov s spletno aplikacijo sem pripravil API-storitev v ogrodju FastAPI. API omogoča, da drugi deli sistema pošljejo sliko obraza in prejmejo rezultat prepoznavne v obliki JSON. S tem model ni omejen samo na lokalno izvajanje prek kamere, ampak ga lahko uporabljajo spletna aplikacija, backend in mobilna aplikacija.

API uporablja funkcije obstoječega ORV modula za zaznavo obraza, predobdelavo slike in napoved uporabnika. Prejeta slika se najprej pretvori v obliko, primerno za OpenCV. Nato sistem poišče obraz, ga normalizira in posreduje naučenemu modelu.

Rezultat obdelave vsebuje podatek o zaznanem obrazu, napovedanega uporabnika, verjetnost napovedi, uporabljen threshold in odločitev, ali je bila napoved sprejeta. Model se hrani v predpomnilniku, zato ga ni treba ponovno nalagati pri vsakem API klicu.

API endpointi

Implementirani so bili naslednje glavni endpointe:

```
@app.get("/")
def root():
    return {
        "service": "Hribovc ORV Face Recognition API",
        "status": "running",
        "endpoints": [
            "GET /health",
            "POST /predict-face",
            "POST /verify-face",
            "POST /phone-preview-frame",
            "POST /phone-preview-close",
        ],
    }
```

GET /health

Endpoint preveri, ali API deluje in ali so datoteke naučenega modela na voljo. Vrne status sistema, lokacijo modela in podatek modelAvailable.

```
@app.get("/health")
def health():
    return {
        "status": "ok",
        "modelDirectory": str(MODEL_DIR),
        "modelAvailable": is_model_available(),
    }
```

POST /predict-face

Endpoint sprejme sliko in vrne napoved uporabnika. Poleg napovedanega uporabnika vrne verjetnost, podatek o zaznanem obrazu in odločitev accepted glede na izbrani threshold.

```
@app.post("/predict-face")
async def predict_face_endpoint(
    image: UploadFile = File(...),
    threshold: float = Form(DEFAULT_THRESHOLD),
    nightMode: bool = Form(False),
):
    threshold = validate_threshold(threshold)
    recognizer = get_recognizer()
    uploaded_image = read_image_from_upload(image)

    prepared_face, face_box = prepare_face_from_image(
        uploaded_image,
        force_night_mode=nightMode,
    )

    if prepared_face is None:
        return {
            "success": False,
            "faceDetected": False,
            "predictedUser": None,
            "probability": 0.0,
            "accepted": False,
            "threshold": threshold,
            "faceBox": None,
            "message": "Obraz ni bil zaznan na poslani sliki.",
        }

    predicted_user, probability = predict_face(prepared_face, recognizer)
    accepted = probability >= threshold

    return {
        "success": True,
        "faceDetected": True,
        "predictedUser": predicted_user,
        "probability": round(float(probability), 4),
        "accepted": bool(accepted),
        "threshold": threshold,
        "faceBox": face_box,
        "message": "Obraz je bil uspešno obdelan.",
    }
```

POST /verify-face

Endpoint se uporablja pri prijavi uporabnika. Poleg slike prejme pričakovano uporabniško ime in preveri, ali se napovedani obraz ujema s pričakovanim uporabnikom. Rezultat vsebuje odločitev *verified*, verjetnost napovedi in podatke o zaznanem obrazu.

```
@app.post("/verify-face")
async def verify_face_endpoint(
    image: UploadFile = File(...),
    expectedUser: str = Form(...),
    threshold: float = Form(DEFAULT_THRESHOLD),
    nightMode: bool = Form(False),
):
    threshold = validate_threshold(threshold)
    uploaded_image = read_image_from_upload(image)

    prepared_face, face_box = prepare_face_from_image(
        uploaded_image,
        force_night_mode=nightMode,
    )

    if prepared_face is None:
        return {
            "success": False,
            "verified": False,
            "faceDetected": False,
            "expectedUser": expectedUser,
            "predictedUser": None,
            "probability": 0.0,
            "threshold": threshold,
            "faceBox": None,
            "message": "Obraz ni bil zaznan na poslani sliki.",
        }

    if expectedUser:
        profile_result = predict_profile_face(prepared_face, expectedUser, threshold)
        predicted_user = profile_result["predictedUser"]
        probability = profile_result["probability"]
        verified = profile_result["verified"]
        margin = profile_result["margin"]
        user_scores = profile_result["userScores"]
    else:
        recognizer = get_recognizer()
        predicted_user, probability = predict_face(prepared_face, recognizer)
        verified = users_match(predicted_user, expectedUser) and probability >= threshold
        margin = None
        user_scores = None

    return {
        "success": True,
        "verified": bool(verified),
        "faceDetected": True,
        "expectedUser": expectedUser,
        "predictedUser": predicted_user,
        "probability": round(float(probability), 4),
        "margin": margin,
        "threshold": threshold,
        "faceBox": face_box,
        "userScores": user_scores,
        "message": "Uporabnik je potrjen." if verified else "Uporabnik ni potrjen.",
    }
```

Validacija in obravnava napak

Dodal sem validacijo thresholda, ki mora biti med 0,0 in 1,0. Če je vrednost neveljavna, API vrne napako 400 z razumljivim opisom. Če model ni na voljo ali ga ni mogoče naložiti, API vrne napako 400.

```
DEFAULT_THRESHOLD = 0.7
MIN_THRESHOLD = 0.0
MAX_THRESHOLD = 1.0
```

```
def validate_threshold(threshold: float) -> float:
    if threshold < MIN_THRESHOLD or threshold > MAX_THRESHOLD:
        raise HTTPException(
            status_code=400,
            detail=f"Threshold mora biti med {MIN_THRESHOLD} in {MAX_THRESHOLD}.",
        )
    return threshold
```

Posebej je obravnavan tudi primer, ko na poslani sliki ni zaznanega obraza. API se v tem primeru ne prekine, ampak vrne odgovor, v katerem so faceDetected, accepted oziroma verified nastavljeni na false.

```
frame = image.copy()
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
detected_face = detect_largest_face(gray, force_night_mode=force_night_mode)
face_box = None
predicted_user = None
probability = 0.0
accepted = False
verified = False
message = "Obraz ni zaznan. Poravnaj obraz v kameri."
box_color = (52, 107, 255)
```

Povezava s spletno aplikacijo

API sem povezal z backendom spletne aplikacije Hribovc. Pri prijavi backend sprejme sliko uporabnika in jo v obliki multipart/form-data pošlje na endpoint /verify-face. Backend nato rezultat ORV API-ja posreduje prijavnemu sistemu.

```

async function verifyFaceWithOrvApi({
  imageBase64,
  expectedUser,
  threshold = ORV_FACE_THRESHOLD,
  nightMode = false,
}) {
  const imageBuffer = imageBase64ToBuffer(imageBase64);

  if (!imageBuffer || imageBuffer.length === 0) {
    throw new Error('Slika za preverjanje obraza ni bila poslana.');
```

Povezava je nastavljiva z okoljskimi spremenljivkami ORV_API_URL, ORV_FACE_THRESHOLD in ORV_FACE_TIMEOUT_MS. S tem lahko spremenimo naslov API-ja, prag zaupanja in najdaljši dovoljen čas čakanja brez spreminjanja programske kode.

```

const ORV_API_URL = normalizeOrvApiUrl(process.env.ORV_API_URL || 'http://localhost:8000');
const ORV_FACE_THRESHOLD = readFaceThreshold(process.env.ORV_FACE_THRESHOLD, 0.7);
const ORV_FACE_TIMEOUT_MS = readPositiveInteger(
  process.env.ORV_FACE_TIMEOUT_MS,
  30000,
  'ORV_FACE_TIMEOUT_MS'
);
const ORV_PC_FACE_LOGIN_ENABLED = process.env.ORV_PC_FACE_LOGIN_ENABLED !== 'false';
const ORV_2FA_TTL_MS = readPositiveInteger(process.env.ORV_2FA_TTL_MS, 90000);
const orv2faChallenges = new Map();
let latestOrv2faChallengeId = null;
```

Docker in enostaven zagon

Za ORV API sem pripravil Dockerfile, ki uporablja Python 3.11. Docker slika namesti sistemske knjižnice za OpenCV, Python dependencyje iz requirements.txt in zažene FastAPI strežnik z Uvicornom na portu 8000.


```

FROM python:3.11-slim

WORKDIR /app

ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1

RUN apt-get update \
    && apt-get install -y --no-install-recommends \
        libgl1 \
        libglu1-mesa \
        libglu1-mesa-dev \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .

RUN pip install --no-cache-dir --upgrade pip \
    && pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["python", "-m", "uvicorn", "api_server:app", "--host", "0.0.0.0", "--port", "8000"]

```

Pripravil sem tudi `.dockerignore`, ki iz Docker slike izključi virtualna okolja, začasne datoteke, predpomnilnik in nepotrebne podatke. S tem je Docker slika manjša in se hitreje zgradi.

```

.venv/
.venv311/
__pycache__/
*.pyc
*.pyo
*.pyd

data/raw/
data/processed/
data/augmented/
data/users/
data/login-attempts.jsonl

.pytest_cache/
.coverage
htmlcov/

.DS_Store
Thumbs.db

```

Za lokalni zagon sem pripravil dve skripti:

- start_orv_api.ps1 za Windows PowerShell,
- start_orv_api.sh za Linux oziroma druga okolja s podporo za shell.

```
param(
    [string]$HostAddress = "0.0.0.0",
    [int]$Port = 8000
)

$ErrorActionPreference = "Stop"

$ScriptDir = Split-Path -Parent $MyInvocation.MyCommand.Path
Set-Location $ScriptDir

$PythonCandidates = @(
    ".\venv311\Scripts\python.exe",
    ".\venv\Scripts\python.exe",
    "python"
)

$PythonCommand = $null

foreach ($Candidate in $PythonCandidates) {
    if ($Candidate -eq "python") {
        $PythonCommand = $Candidate
        break
    }

    if (Test-Path $Candidate) {
        $PythonCommand = $Candidate
        break
    }
}

Write-Host "[ORV] Zagon ORV API streznika"
Write-Host "[ORV] Delovna mapa: $ScriptDir"
Write-Host "[ORV] Python: $PythonCommand"
Write-Host "[ORV] URL: http://localhost:$Port"
Write-Host ""

& $PythonCommand -m uvicorn api_server:app --host $HostAddress --port $Port
```

```
#!/usr/bin/env bash
set -euo pipefail

cd "$(dirname "$0")"

HOST="{HOST:-0.0.0.0}"
PORT="{PORT:-8000}"

if [ -x ".venv/bin/python" ]; then
    PYTHON_COMMAND=".venv/bin/python"
elif [ -x ".venv311/bin/python" ]; then
    PYTHON_COMMAND=".venv311/bin/python"
elif command -v python3 >/dev/null 2>&1; then
    PYTHON_COMMAND="python3"
else
    PYTHON_COMMAND="python"
fi

echo "[ORV] Zagon ORV API streznika"
echo "[ORV] Delovna mapa: $(pwd)"
echo "[ORV] Python: $PYTHON_COMMAND"
echo "[ORV] URL: http://localhost:$PORT"
echo ""

exec "$PYTHON_COMMAND" -m uvicorn api_server:app --host "$HOST" --port "$PORT"
```

Obe skripti nastavita delovno mapo in zaženeta API strežnik na izbranem naslovu in portu.

Testiranje API-ja

Za preverjanje delovanja API-ja sem pripravil avtomatizirane teste z orodjem pytest in FastAPI TestClient. Testi preverjajo:

- delovanje endpointa /health,

```
import sys
from pathlib import Path

from fastapi.testclient import TestClient

sys.path.append(str(Path(__file__).resolve().parent.parent))

import api_server

client = TestClient(api_server.app)

def test_health_returns_status_and_model_information():
    response = client.get("/health")

    assert response.status_code == 200

    body = response.json()

    assert body["status"] == "ok"
    assert "modelDirectory" in body
    assert "modelAvailable" in body
    assert isinstance(body["modelAvailable"], bool)
```

- zavrnitev thresholda, večjega od 1,
- zavrnitev thresholda, manjšega od 0,

```
import sys
import numpy as np
from pathlib import Path

sys.path.append(str(Path(__file__).resolve().parent.parent))
from svm_face_recognition import predict_user

THRESHOLD = 0.7

def test_threshold():
    print(f"[TEST] Testiram threshold verjetnosti (prag: {THRESHOLD})...")

    test_dir = Path(__file__).resolve().parent.parent / "data" / "test_images"
    users = ["anja", "Maja", "Ziga"]

    total = 0
    above_threshold = 0
    below_threshold = 0

    import cv2
    face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')

    for user in users:
        user_dir = test_dir / user
        if not user_dir.exists():
            continue
        for img_path in user_dir.glob("*.jpg"):
            img = cv2.imread(str(img_path))
            if img is None:
                continue
            gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
            faces = face_cascade.detectMultiScale(gray, 1.1, 4)
            if len(faces) == 0:
                continue
            x, y, w, h = faces[0]
            face = cv2.resize(gray[y:y+h, x:x+w], (128, 128)).astype(np.float32) / 255.0
            username, prob = predict_user(face)
            total += 1
            if prob >= THRESHOLD:
                above_threshold += 1
            else:
                below_threshold += 1
            print(f"[NIZKA ZAPUSTOST] {img_path.name}: {username} (verjetnost: {prob:.2f})")

    print(f"\n[TEST] Nad pragom: {above_threshold}/{total}")
    print(f"[TEST] Pod pragom: {below_threshold}/{total}")

if __name__ == "__main__":
    test_threshold()
```

```
import sys
from pathlib import Path

import cv2
import numpy as np
from fastapi.testclient import TestClient

sys.path.append(str(Path(__file__).resolve().parent.parent))

import api_server

client = TestClient(api_server.app)

def make_test_image_bytes():
    image = np.zeros((128, 128, 3), dtype=np.uint8)
    success, encoded = cv2.imencode(".jpg", image)

    assert success

    return encoded.tobytes()

def test_predict_face_rejects_threshold_above_one():
    response = client.post(
        "/predict-face",
        files={
            "image": ("face.jpg", make_test_image_bytes(), "image/jpeg"),
        },
        data={
            "threshold": "1.5",
            "nightMode": "false",
        },
    )

    assert response.status_code == 400
    assert response.json()["detail"] == "Threshold mora biti med 0.0 in 1.0."

def test_verify_face_rejects_threshold_below_zero():
    response = client.post(
        "/verify-face",
        files={
            "image": ("face.jpg", make_test_image_bytes(), "image/jpeg"),
        },
        data={
            "expectedUser": "anja",
            "threshold": "-0.1",
            "nightMode": "false",
        },
    )

    assert response.status_code == 400
    assert response.json()["detail"] == "Threshold mora biti med 0.0 in 1.0."
```

- pravilen odgovor endpointa /predict-face, ko obraz ni zaznan,
- pravilen odgovor endpointa /verify-face, ko obraz ni zaznan.

```
import sys
from pathlib import Path

import cv2
import numpy as np
from fastapi.testclient import TestClient

sys.path.append(str(Path(__file__).resolve().parent.parent))

import api_server

client = TestClient(api_server.app)

> def make_test_image_bytes(): ...

> def test_predict_face_returns_no_face_response(monkeypatch): ...

> def test_verify_face_returns_no_face_response(monkeypatch): ...
```